



BÀI GIẢNG

TÍNH TOÁN MỀM

SOFT COMPUTING

Giảng viên: Nguyễn Ngọc Sơn

Email: nguyenngocson@iuh.edu.vn

Chương 2. Thuật toán tối ưu gián tiếp

Trong chương này, chúng ta sẽ học:

- Tối ưu dựa trên đạo hàm (GD, Newton, Adam).
- Hạn chế của các phương pháp cổ điển.
- Vai trò của tối ưu trong học máy.

Chương 2. Thuật toán tối ưu gián tiếp

2.1 Tối ưu hóa

2.2 Phương pháp bậc nhất

2.3 Phương pháp bậc hai

2.4 Tối ưu hóa trong học máy

Chương 2. Thuật toán tối ưu gián tiếp

2.1 Tối ưu hóa

- ❖ Hàm mục tiêu
- ❖ Biến quyết định
- ❖ Lời giải của bài toán tối ưu
- ❖ Không gian quyết định
- ❖ Tối ưu cục bộ, toàn cục và cận tối ưu
- ❖ Phân loại thuật toán tối ưu

2.1. Tối ưu hóa

❖ Động cơ của tối ưu hóa

- Các kỹ sư thường phải thiết kế và vận hành hệ thống nhằm đạt mục tiêu đề ra trong điều kiện nhiều ràng buộc.
- Tối ưu hóa là quá trình tìm kiếm có hệ thống các phương án thiết kế và vận hành để xác định tập hợp hành động mang lại hiệu quả tối ưu cho hệ thống.

Ví dụ: Thiết kế động cơ điện cho Drone

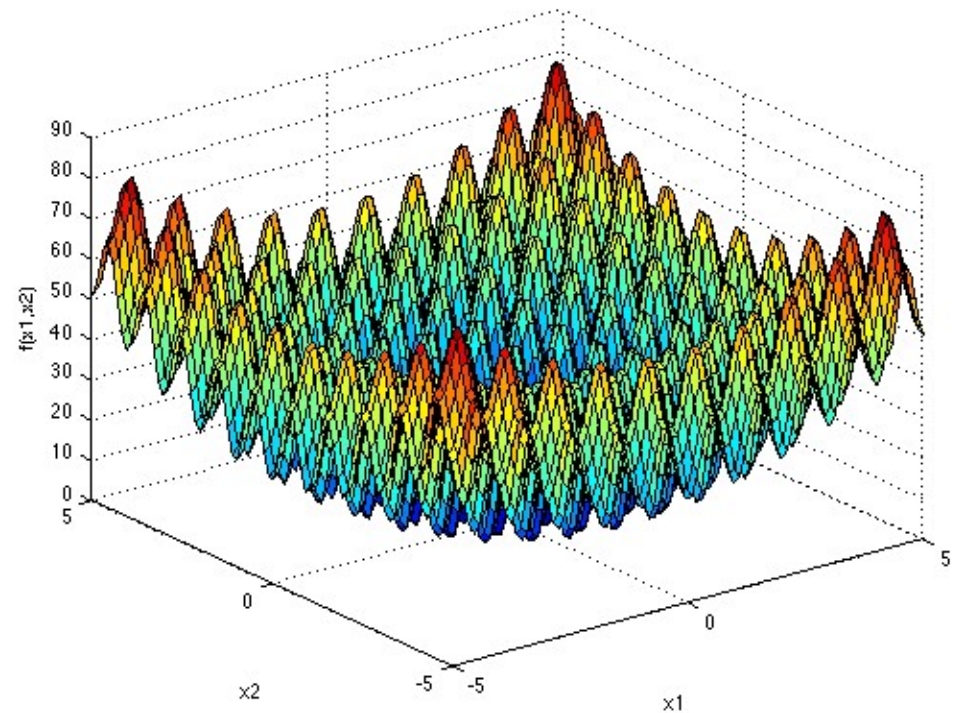
- Mục tiêu: **hiệu suất cao**
- Ràng buộc: **chi phí, kích thước, nhiệt độ**
- Bài toán: chọn thông số thiết kế tối ưu



2.1. Tối ưu hóa

❖ Định nghĩa tối ưu hóa

Tối ưu hóa là quá trình tìm kiếm có hệ thống nhằm tìm giá trị lớn nhất hoặc nhỏ nhất của hàm mục tiêu trong không gian khả thi



2.1. Tối ưu hóa

❖ Bài toán tối ưu hóa đơn mục tiêu

Tối ưu hóa hàm $f(X), X = [x_1, x_2, \dots, x_N]$

$$g_j(X) < b_j, \quad j = 1, 2, \dots, m$$

$$x_i^{(L)} \leq x_i \leq x_i^{(U)}, \quad i = 1, 2, \dots, N$$

- ✓ $f(X)$ là hàm mục tiêu;
- ✓ X là tập các biến quyết định x_i tạo thành lời giải khả thi cho bài toán tối ưu; x_i là biến quyết định thứ i ;
- ✓ N là số biến hay số chiều của bài toán tối ưu;
- ✓ $g_j(X)$ ràng buộc thứ j ; b_j hằng số ràng buộc thứ j ; m tổng các ràng buộc;
- ✓ $x_i^{(L)}$ là cận dưới; $x_i^{(U)}$ là cận trên của biến quyết định thứ i .

2.1. Tối ưu hóa

❖ Hàm mục tiêu – Objective / Loss function

- Hàm mục tiêu là hàm cần xác định giá trị lớn nhất hoặc giá trị nhỏ nhất bằng cách chọn các biến quyết định thỏa mãn tất cả các ràng buộc của bài toán tối ưu.

2.1. Tối ưu hóa

❖ Biến quyết định

- Biến quyết định xác định giá trị của hàm mục tiêu.
- Bài toán tối ưu trong đó các biến quyết định nhận giá trị liên tục trong một khoảng xác định được gọi là bài toán **tối ưu liên tục**.
- Bài toán tối ưu trong đó các biến quyết định chỉ nhận các giá trị rời rạc trong một tập xác định được gọi là bài toán **tối ưu rời rạc**.

2.1. Tối ưu hóa

❖ Lời giải của bài toán tối ưu

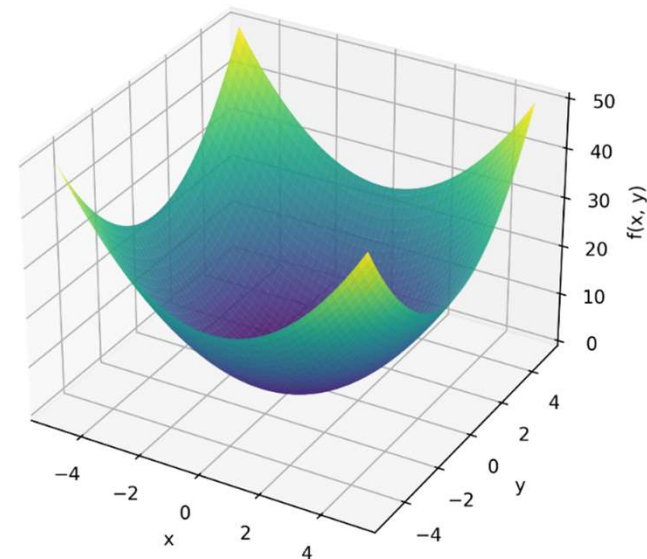
- Mỗi hàm mục tiêu được mô tả dưới dạng các biến quyết định.
 - Khi có một biến quyết định → bài toán tối ưu một chiều.
 - Khi có N biến quyết định → bài toán tối ưu N chiều.
- Trong bài toán tối ưu N chiều, mỗi lời giải là một vector gồm N biến quyết định.

2.1. Tối ưu hóa

❖ Không gian quyết định

- Tập hợp các biến quyết định thỏa mãn các ràng buộc của bài toán tối ưu được gọi là không gian quyết định khả thi.
- Mỗi nghiệm là một vector N chiều.
- Thuật toán tối ưu tìm nghiệm tốt nhất trong không gian này.

Hình mô tả không gian quyết định 2 chiều với các biến quyết định x và y của hàm mục tiêu $f(x,y) = x^2 + y^2$, trong đó $x, y \in [-5,5]$.



2.1. Tối ưu hóa

❖ Ràng buộc

- Các ràng buộc giới hạn các giá trị có thể của biến quyết định, chẳng hạn dưới dạng giới hạn trên, dưới hoặc các mối quan hệ toán học giữa các biến.
- Mục tiêu của bài toán tối ưu là tìm một nghiệm khả thi thỏa mãn tất cả các ràng buộc và đạt giá trị tốt nhất của hàm mục tiêu. Do đó, lời giải tối ưu luôn nằm trong không gian quyết định khả thi.

2.1. Tối ưu hóa

❖ Toàn cục – global optimum và cục bộ - local optima

Tối ưu cục bộ được định nghĩa là một lời giải tốt nhất của hàm mục tiêu trong vùng lân cận nó. Trong bài toán tối ưu một chiều, biến quyết định X^* là một cực tiểu cục bộ sẽ thỏa mãn điều kiện:

$$f(X^*) \leq f(X), \quad X^* - \varepsilon \leq X \leq X^* + \varepsilon$$

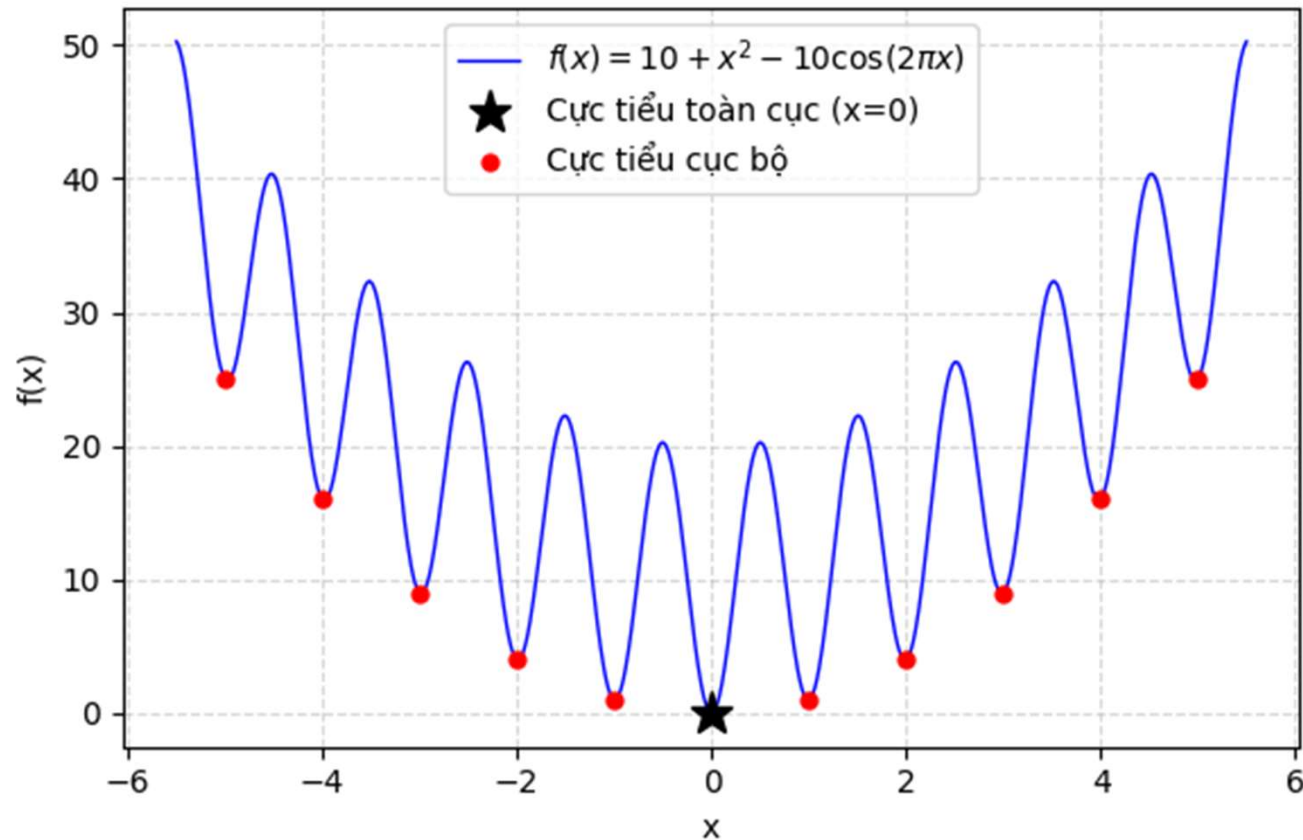
Tối ưu toàn cục được định nghĩa là một lời giải tốt nhất của hàm mục tiêu trong toàn bộ không gian quyết định. Với tối ưu một chiều, biến quyết định X^* là một cực tiểu toàn cục sẽ thỏa mãn điều kiện:

$$f(X^*) \leq f(X)$$

Trong đó $\varepsilon > 0$.

2.1. Tối ưu hóa

❖ Tối ưu toàn cục và cục bộ

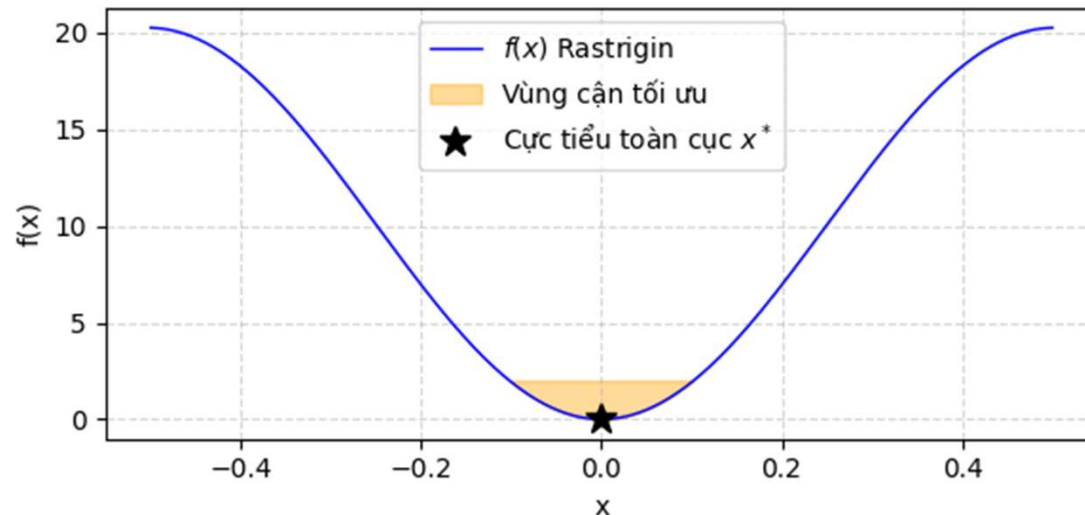


Hình minh họa tối ưu cục bộ và toàn cục của bài toán tối ưu trong không gian một chiều

2.1. Tối ưu hóa

❖ Cận tối ưu

- Điểm cận tối ưu là điểm có giá trị hàm mục tiêu rất gần, nhưng không nhất thiết bằng, giá trị tối ưu toàn cục.



- Trong nhiều bài toán kỹ thuật phức tạp, việc đạt được tối ưu toàn cục là khó hoặc không khả thi do giới hạn của mô hình, phương pháp giải và tài nguyên tính toán.

2.1. Tối ưu hóa

❖ Phương pháp giải bài toán tối ưu hóa

Phương pháp

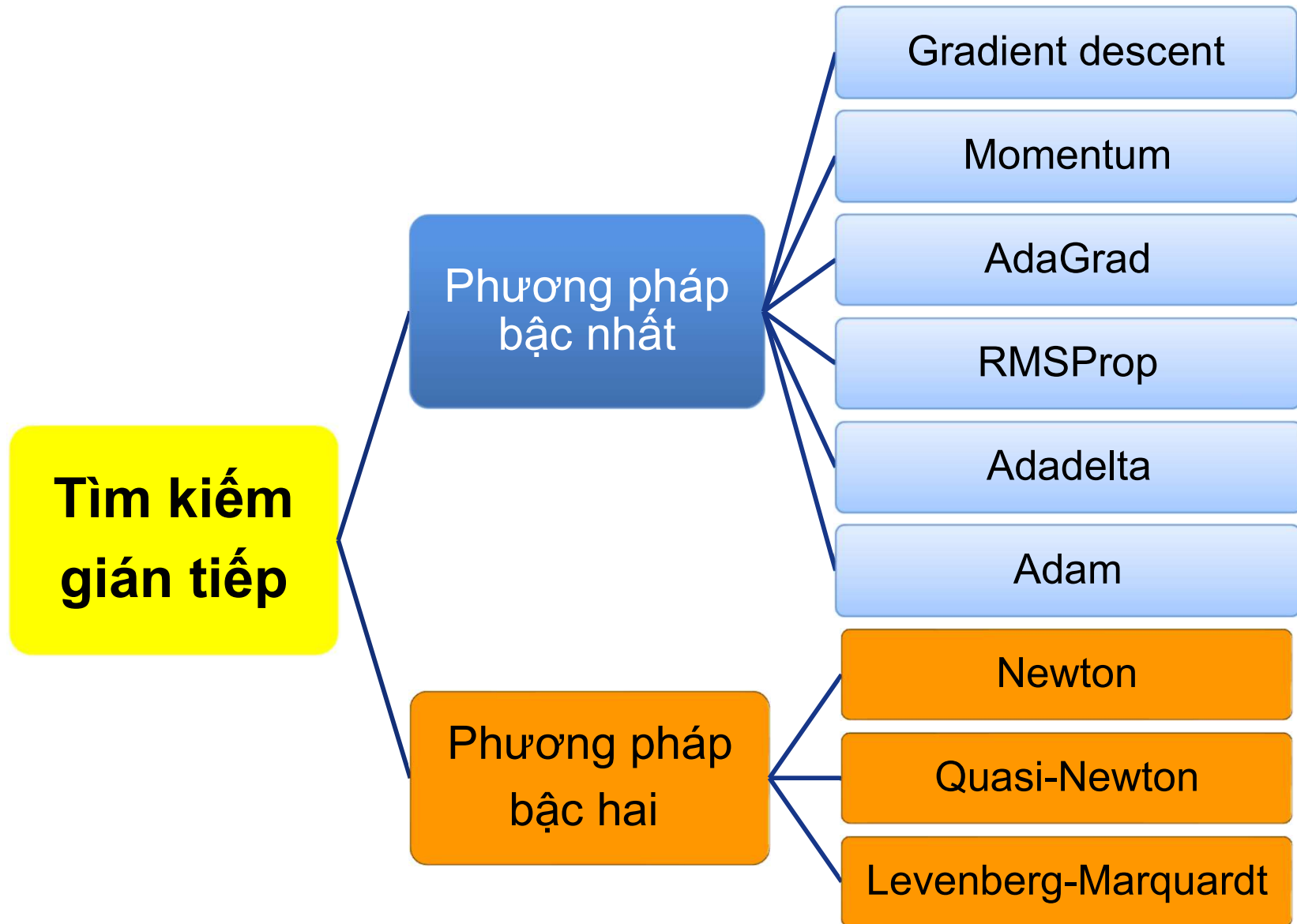
Tìm kiếm trực tiếp

- Dựa trên thông tin đạo hàm
- Hội tụ nhanh, nhưng dễ kẹt cực trị cục bộ
- Hạn chế với bài toán phi tuyến mạnh, rời rạc

Tìm kiếm gián tiếp

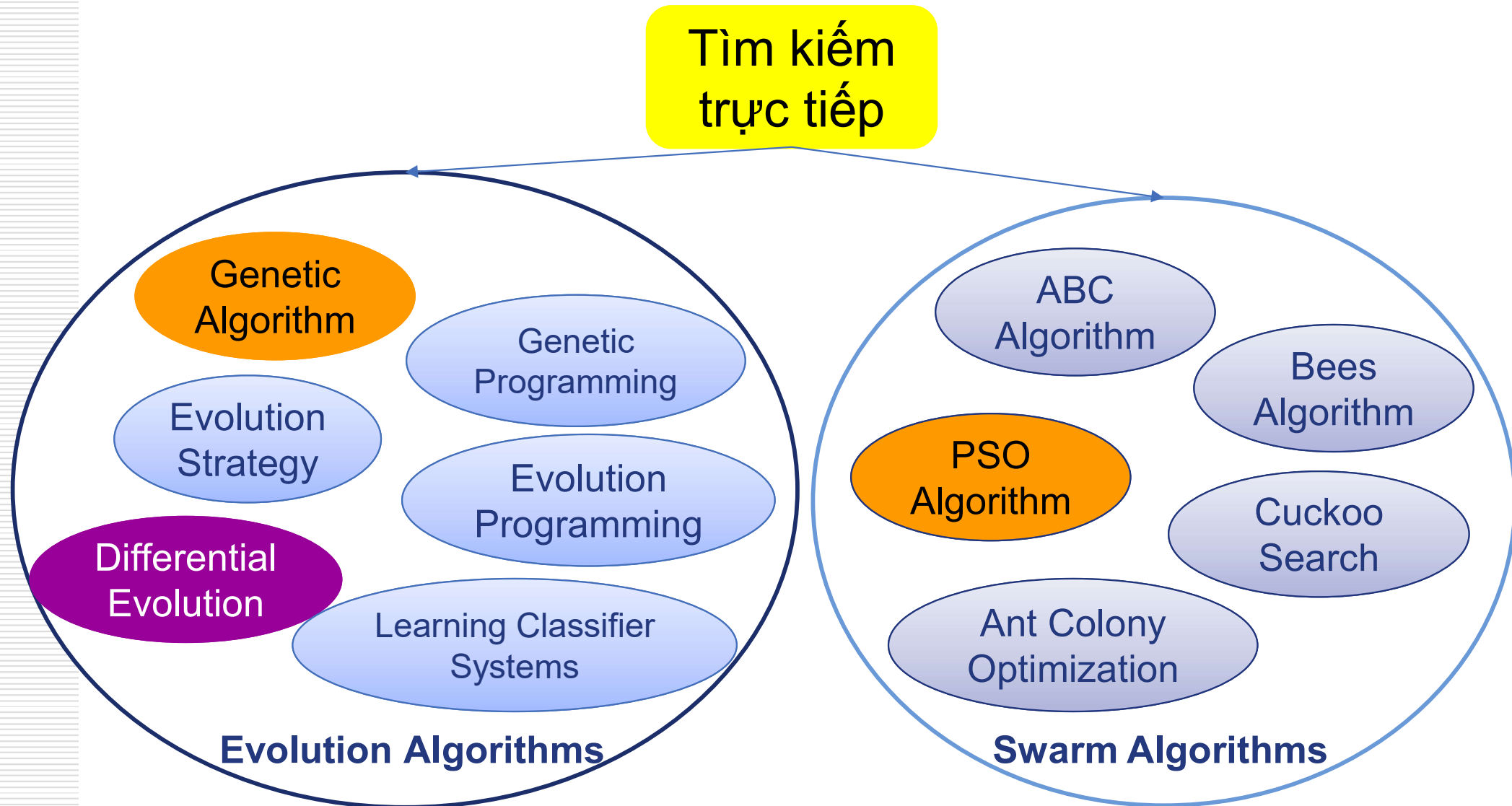
- Dựa trên giá trị hàm mục tiêu và ràng buộc, không cần thông tin đạo hàm.
- Nghiệm tối ưu toàn cục, nhưng chi phí tính toán cao.
- Áp dụng cho bài toán phi tuyến, rời rạc.

2.1. Tối ưu hóa



2.1. Tối ưu hóa

Tìm kiếm
trực tiếp



Chương 2. Thuật toán tối ưu gián tiếp

2.2 Phương pháp bậc nhất

2.2.1 Thuật toán suy giảm độ dốc GD

2.2.2 Thuật toán Momentum

2.2.3 Thuật toán AdaGrad

2.2.4 Thuật toán RMSProp

2.2.5 Thuật toán Adadelata

2.2.6 Thuật toán Adam

2.2.1 Thuật toán tối ưu GD

❖ Ý tưởng

- ✓ Gradient Descent (GD) là một trong những thuật toán tối ưu cơ bản và phổ biến nhất, được sử dụng để tìm điểm cực tiểu của hàm mục tiêu thông qua việc cập nhật tham số theo hướng ngược với gradient.
- ✓ Gradient của hàm mục tiêu tại một điểm cho biết hướng tăng nhanh nhất của hàm. Do đó, để giảm giá trị hàm mục tiêu, ta di chuyển theo hướng ngược lại của gradient.

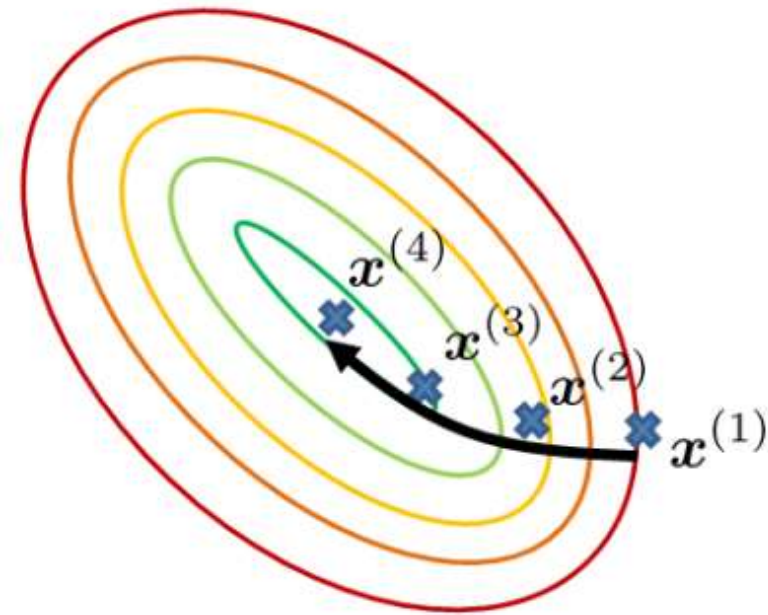
2.2.1 Thuật toán tối ưu GD

❖ Luật cập nhật

Giải thuật GD được mô tả như sau khi tìm cực tiểu của hàm $f(X)$:

$$X(t+1) = X(t) - \alpha * \nabla f(X(t)) \quad t = 0, 1, 2, 3, \dots$$

- ✓ α là *learning rate* (tốc độ học), $\alpha > 0$
- ✓ t là số vòng lặp
- ✓ X : véc-tơ các biến.
- ✓ $\nabla(nabla)$: đạo hàm riêng của $f(X)$



2.2.1 Thuật toán tối ưu GD

❖ Điều kiện dừng của thuật toán GD

$$\|\nabla f(x_t)\| < \varepsilon \text{ hoặc } t \geq \text{MaxIter}$$

- ε là một số dương nhỏ hay gọi là ngưỡng hội tụ (ví dụ 10^{-5}).
- *MaxIter*: Số vòng lặp tối đa.
- t : chỉ số vòng lặp.
- $\|\nabla f(x_t)\|$: Norm chuẩn 2

2.2.1 Thuật toán tối ưu GD

❖ Pseudocode

Đầu vào

- Hàm mục tiêu $f(x)$
- Giá trị ban đầu x_0
- Hệ số α
- Ngưỡng hội tụ ε
- MaxIter số vòng lặp tối đa

Bắt đầu

1. Khởi tạo x_0 , δ , ε và MaxIter
2. While ($\|\nabla f(x_t)\| \geq \varepsilon$ và số vòng lặp chưa đạt MaxIter)
 - a). Tính gradient: $\nabla f(x_t)$.
 - b). Cập nhật tham số: $x_{t+1} = x_t - \alpha * \nabla f(x_t)$
3. End
4. Trả về giá trị x tối ưu

Kết thúc

2.2.1 Thuật toán tối ưu GD

❖ Đánh giá thuật toán GD dựa trên chỉ số thống kê

- Mục đích

- ✓ GD phụ thuộc vào điểm khởi tạo và learning rate, kết quả có thể khác nhau giữa các lần chạy.

- Các chỉ số sử dụng

- ✓ Min: giá trị hàm mục tiêu tốt nhất.
- ✓ Max: giá trị kém nhất.
- ✓ Mean: hiệu quả trung bình.
- ✓ Std: độ ổn định của thuật toán.

- Nhận xét

- ✓ Mean nhỏ → GD hoạt động hiệu quả.
- ✓ Std nhỏ → GD ổn định với khởi tạo khác nhau.

2.2.1 Thuật toán tối ưu GD

❖ Đánh giá thuật toán GD dựa trên chỉ số thống kê

Giả sử thuật toán GD được chạy N lần, thu được các giá trị hàm mục tiêu:

$$f_1, f_2, \dots, f_N$$

- Giá trị nhỏ nhất (Min) $\text{Min} = \min_{i=1, \dots, N} f_i$

- Giá trị lớn nhất (Max) $\text{Max} = \max_{i=1, \dots, N} f_i$

- Giá trị trung bình (Mean)

$$\text{Mean} = \frac{1}{N} \sum_{i=1}^N f_i$$

- Độ lệch chuẩn (Std)

$$\text{Std} = \sqrt{\frac{1}{N} \sum_{i=1}^N (f_i - \text{Mean})^2}$$

2.2.1 Thuật toán tối ưu GD

❖ Phân loại Gradient Descent

Gradient Descent có thể được phân loại dựa trên cách sử dụng dữ liệu trong mỗi lần cập nhật tham số. Ba biến thể phổ biến gồm:

▪ Batch Gradient Descent

- ✓ Sử dụng toàn bộ tập dữ liệu trong mỗi lần cập nhật.
- ✓ Cập nhật tham số sau mỗi epoch.
- ✓ Gradient ổn định, hội tụ tốt.
- ✓ Tốn bộ nhớ, chậm với dữ liệu lớn.
- ✓ Dễ kết cực tiểu cục bộ.

2.2.1 Thuật toán tối ưu GD

❖ Phân loại Gradient Descent

Gradient Descent có thể được phân loại dựa trên cách sử dụng dữ liệu trong mỗi lần cập nhật tham số. Ba biến thể phổ biến gồm:

▪ Stochastic Gradient Descent (SGD)

- ✓ Cập nhật tham số sau mỗi mẫu dữ liệu.
- ✓ Tiết kiệm bộ nhớ.
- ✓ Hội tụ nhanh hơn nhưng gradient nhiễu.
- ✓ Có thể thoát cực tiểu cục bộ.
- ✓ Đường hội tụ dao động.

2.2.1 Thuật toán tối ưu GD

❖ Phân loại Gradient Descent

Gradient Descent có thể được phân loại dựa trên cách sử dụng dữ liệu trong mỗi lần cập nhật tham số. Ba biến thể phổ biến gồm:

▪ Mini-Batch Gradient Descent

- ✓ Chia dữ liệu thành các mini-batch.
- ✓ Cập nhật sau mỗi mini-batch.
- ✓ Cân bằng giữa tốc độ và độ ổn định.
- ✓ Phổ biến nhất trong học máy hiện đại.
- ✓ Kích thước batch cần chọn phù hợp.

2.2.1 Thuật toán tối ưu GD

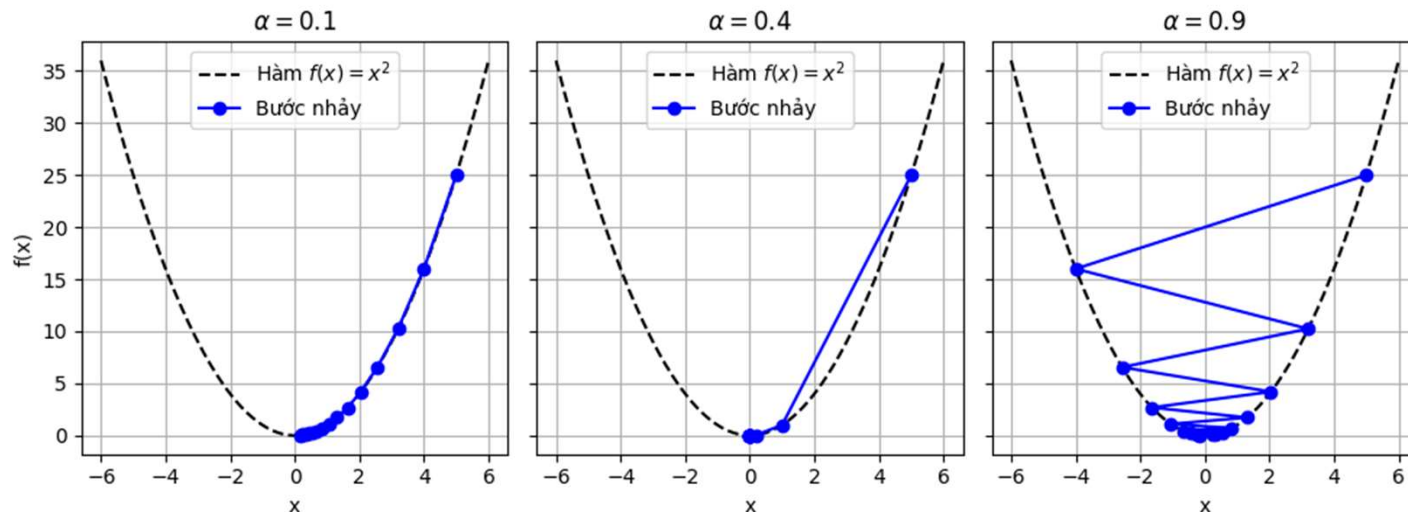
❖ Các thông số ảnh hưởng đến chất lượng GD

■ Tốc độ học

- ✓ Nếu α quá nhỏ, thuật toán sẽ phải thực hiện nhiều bước để hội tụ và sẽ mất nhiều thời gian.
- ✓ Nếu α quá lớn sẽ khiến thuật toán đi qua cực tiểu, và vượt hẳn ra ngoài khiến thuật toán không thể hội tụ được.

■ Giá trị khởi tạo ban đầu

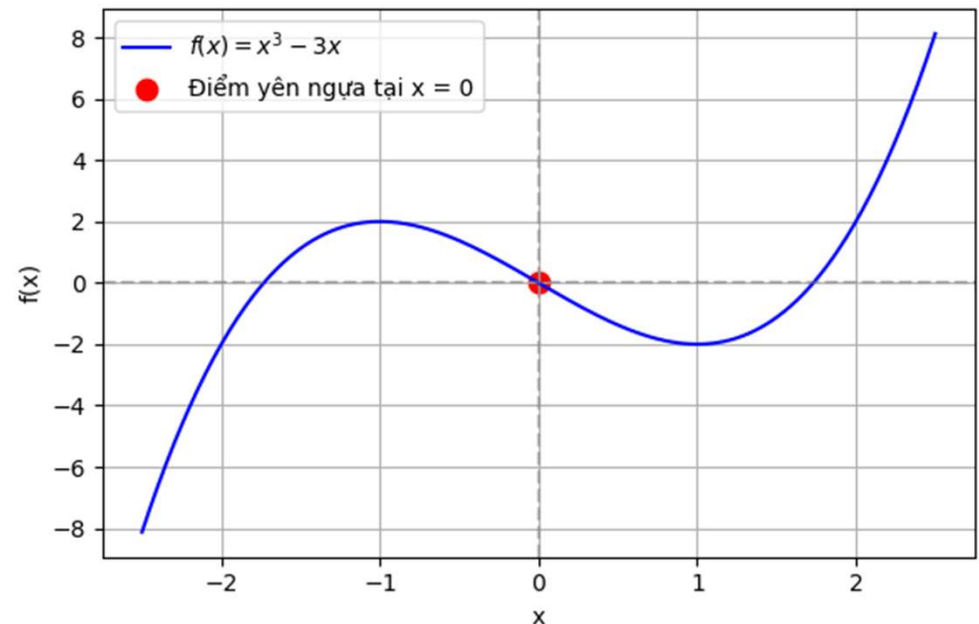
- ✓ Nếu khởi tạo ban đầu gần điểm cực tiểu thì sẽ cần ít bước lặp để hội tụ và ngược lại.



2.2.1 Thuật toán tối ưu GD

❖ Hạn chế GD

- **Cực tiểu cục bộ (Local minimum):** Là điểm mà giá trị hàm mục tiêu nhỏ nhất trong một lân cận, với độ dốc tăng lên ở cả hai phía của điểm đó.
- **Điểm yên ngựa (Saddle point):** Là điểm mà gradient bằng 0, nhưng không phải là cực tiểu hay cực đại. Tại điểm này, hàm mục tiêu giảm theo một số hướng và tăng theo các hướng khác.
- **Chi phí tính toán cao với tập dữ liệu lớn:** Với các tập dữ liệu lớn, việc tính toán gradient trên toàn bộ dữ liệu trong mỗi lần cập nhật có thể tốn nhiều tài nguyên.



2.2.1 Thuật toán tối ưu GD

❖ **Ví dụ 1** Tìm cực tiểu của hàm số

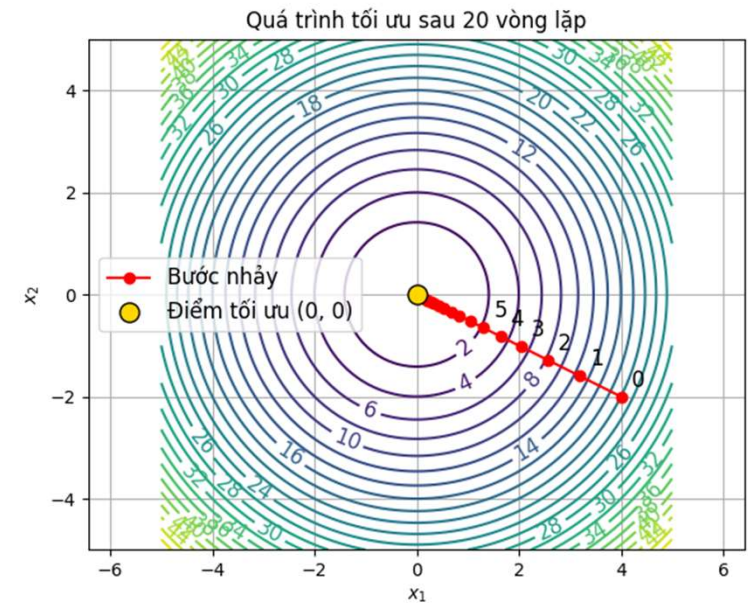
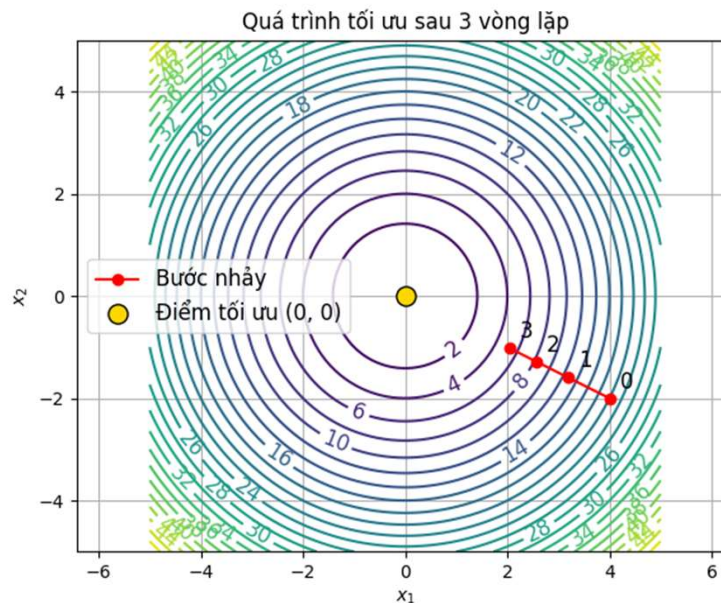
$$f(x_1, x_2) = x_1^2 + x_2^2, \text{ trong đó } x_1 \in [-5, 5] \text{ và } x_2 \in [-5, 5]$$

Gợi ý giải tay:

2.2.1 Thuật toán tối ưu GD

❖ **Ví dụ 1** $f(x_1, x_2) = x_1^2 + x_2^2$, trong đó $x_1 \in [-5, 5]$ và $x_2 \in [-5, 5]$

Chạy code:



Nhận xét:

- ✓ Các đường tròn biểu diễn các đường đồng mức của hàm mục tiêu.
- ✓ Ở giai đoạn đầu, bước cập nhật lớn do còn xa điểm tối ưu.
- ✓ Khi tiến gần điểm tối ưu, bước cập nhật giảm dần, thể hiện quá trình hội tụ của thuật toán.

2.2.2 Thuật toán Momentum

❖ Ý tưởng:

- ✓ GD chỉ dùng gradient hiện tại $\rightarrow$ dễ dao động.
- ✓ Momentum thêm quán tính (vận tốc) để tăng tốc hội tụ.

❖ Công thức cập nhật:

$$\begin{aligned} \mathbf{v}_{t+1} &= \beta \mathbf{v}_t + (1 - \beta) \nabla f(\mathbf{x}_t) \\ \mathbf{x}_{t+1} &= \mathbf{x}_t - \alpha \mathbf{v}_{t+1} \end{aligned}$$

- ✓ $\mathbf{x}_t$ tham số tại bước t .
- ✓ $\nabla f(\mathbf{x}_t)$ gradient của hàm mục tiêu.
- ✓ $\mathbf{v}_t$ vận tốc (momentum).
- ✓ α là *learning rate* (tốc độ học), $\alpha > 0$.
- ✓ $\beta \in [0,1)$ hệ số momentum (thường $\beta = 0.9$).

2.2.2 Thuật toán Adagrad

❖ Ý tưởng:

- ✓ AdaGrad điều chỉnh tốc độ học riêng cho từng tham số dựa trên lịch sử gradient.
- ✓ Các tham số có gradient lớn sẽ có tốc độ học giảm dần, trong khi các tham số có gradient nhỏ được cập nhật với tốc độ học lớn hơn..

❖ Tính toán tổng bình phương gradient

$G_t = G_{t-1} + (\nabla f(\mathbf{x}_t))^2$ G_t là tổng bình phương các gradient trước đó.

❖ Cập nhật

$$x_{t+1} = x_t - \alpha_t \nabla f(\mathbf{x}_t)$$

$$\alpha_t = \frac{\alpha}{\sqrt{G_t + \epsilon}}$$

- ✓ α là *learning rate*
- ✓ $\epsilon > 0$ hằng số nhỏ để tránh chia 0

❖ Nhận xét

- ✓ AdaGrad phù hợp với bài toán dữ liệu thưa (sparse data).
- ✓ Nhược điểm: tốc độ học giảm quá nhanh, có thể làm dừng sớm.

2.2.3 Thuật toán RMSProp

❖ Ý tưởng:

- ✓ RMSProp cải tiến AdaGrad bằng cách sử dụng trung bình trượt mũ của bình phương gradient, giúp tránh hiện tượng tốc độ học giảm quá nhanh.

❖ Cập nhật trung bình bình phương gradient

$$G_t = \rho G_{t-1} + (1 - \rho)(\nabla f(\mathbf{x}_t))^2 \quad \checkmark \quad \rho \in [0,1) \text{ hệ số suy giảm } (\rho = 0.9).$$

❖ Cập nhật

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha_t \nabla f(\mathbf{x}_t)$$

$$\alpha_t = \frac{\alpha}{\sqrt{G_t + \epsilon}}$$

- ✓ α là *learning rate*
- ✓ $\epsilon > 0$ hằng số nhỏ để tránh chia 0

❖ Nhận xét

- ✓ RMSProp cải thiện AdaGrad nhưng thường kém ổn định hơn Adam.

2.2.4 Thuật toán Adadelta

❖ Ý tưởng:

- ✓ Adadelta được đề xuất nhằm khắc phục nhược điểm của AdaGrad, đó là tốc độ học giảm quá nhanh, khiến quá trình huấn luyện chậm lại hoặc dừng sớm.
- ✓ AdaDelta sử dụng trung bình động suy giảm theo hàm mũ của bình phương gradient và của các bước cập nhật tham số, từ đó tự động điều chỉnh tốc độ học mà không cần chỉ định trước learning rate cố định

❖ Nhận xét

- ✓ AdaDelta không yêu cầu lựa chọn learning rate ban đầu.
- ✓ So với AdaGrad, AdaDelta duy trì tốc độ học ổn định hơn trong các giai đoạn huấn luyện kéo dài.

2.2.4 Thuật toán Adadelta

❖ B1. Tính trung bình động của bình phương gradient

$$G_t = \rho G_{t-1} + (1 - \rho)(\nabla f(\mathbf{x}_t))^2$$

✓ $\rho \in [0,1)$ hệ số suy giảm ($\rho = 0.9$).

❖ B2. Tính trung bình động của bình phương bước cập nhật

$$E_t = \gamma E_{t-1} + (1 - \gamma)(\Delta x(t))^2$$

✓ $\Delta x_t = x_t - x_{t-1}$ là bước cập nhật tham số tại thời điểm

❖ B3. Tính hệ số tốc độ học thích nghi

$$\alpha_{adadelta} = \frac{\sqrt{E_t}}{\sqrt{S_t} + \epsilon}$$

❖ B4. Cập nhật

$$x_{t+1} = x_t - \alpha_{adadelta} \nabla f(\mathbf{x}_t)$$

✓ $\epsilon > 0$ hằng số nhỏ

2.2.5 Thuật toán Adam

❖ Ý tưởng

- ✓ Adam (Adaptive Moment Estimation), là một thuật toán tối ưu hóa dựa trên gradient, kết hợp ưu điểm của Momentum và RMSProp, cho phép điều chỉnh tốc độ học thích nghi theo từng tham số trong quá trình huấn luyện.
- ✓ Thuật toán đồng thời ước lượng moment bậc nhất (trung bình động của gradient) và moment bậc hai (trung bình động bình phương gradient), từ đó cải thiện tốc độ hội tụ và tính ổn định.

❖ Nhận xét

- ✓ Adam hội tụ nhanh và ổn định trong các bài toán có gradient nhiễu hoặc gradient thưa.
- ✓ Thuật toán ít nhạy cảm với việc lựa chọn siêu tham số.
- ✓ Adam đặc biệt phù hợp cho các mô hình học sâu với số lượng tham số lớn và quá trình huấn luyện kéo dài.

2.2.4 Thuật toán Adam

❖ B1. Ước lượng moment bậc nhất (Momentum)

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla f(\mathbf{x}_t)$$

✓ $\beta \in [0,1)$ hệ số suy giảm ($\beta = 0.9$).

❖ B2. Ước lượng moment bậc hai (RMSProp)

$$G_t = \rho G_{t-1} + (1 - \rho) (\nabla f(\mathbf{x}_t))^2$$

✓ $\rho \in [0,1)$ hệ số suy giảm ($\rho = 0.9$).

❖ B3. Hiệu chỉnh sai lệch

$$\hat{v}_t = \frac{v_t}{1 - \beta^t}$$

$$\hat{G}_t = \frac{G_t}{1 - \rho^t}$$

❖ B4. Cập nhật

$$x_{t+1} = x_t - \alpha \frac{\hat{v}_t}{\sqrt{\hat{G}_t} + \epsilon}$$

✓ α là *learning rate*

✓ $\epsilon > 0$ hằng số nhỏ

Chương 2. Thuật toán tối ưu gián tiếp

2.3 Phương pháp bậc hai

2.3.1 Thuật toán Newton

2.3.2 Thuật toán Quasi-Newton

2.3.3 Thuật toán Levenberg-Marquardt

Chương 2. Thuật toán tối ưu gián tiếp

2.3 Phương pháp bậc hai

Phương pháp bậc hai (Second-order method) là các phương pháp sử dụng thông tin từ đạo hàm bậc hai (ma trận Hessian) của hàm mục tiêu để cải thiện việc tìm kiếm lời giải tối ưu so với các phương pháp bậc nhất.

Các phương pháp này thường có khả năng hội tụ nhanh hơn so với các phương pháp bậc nhất, nhưng đòi hỏi chi phí tính toán cao hơn, đặc biệt là khi không gian tham số lớn.

2.3.1 Thuật toán Newton

❖ Ý tưởng:

- ✓ Phương pháp Newton là kỹ thuật tối ưu hóa sử dụng đạo hàm bậc hai để tìm cực tiểu hoặc cực đại của hàm mục tiêu;
- ✓ Gradient xác định hướng di chuyển, còn thông tin bậc hai cho phép xấp xỉ Taylor bậc hai nhằm ước lượng bước nhảy phù hợp và hội tụ nhanh hơn.

❖ Cập nhật

$$x_{t+1} = x_t - H^{-1} \nabla f(x_t)$$

- ✓ $\nabla f(x_t)$ là gradient tại x_t , biểu thị hướng di chuyển.
- ✓ $H = \nabla^2 f(x_t)$ là ma trận đạo hàm riêng bậc 2 của f (ma trận Hessian).
- ✓ H^{-1} là ma trận nghịch đảo giúp điều chỉnh kích thước bước nhảy.

2.3.1 Thuật toán Newton

❖ Pseudocode

Đầu vào

- Hàm mục tiêu $f(x)$
- Giá trị ban đầu x_0
- Ngưỡng hội tụ ε
- MaxIter số vòng lặp tối đa

Bắt đầu

1. Khởi tạo giá trị ban đầu x_0 , ε và MaxIter
2. While ($\|\nabla f(x_t)\| \geq \varepsilon$ và số vòng lặp chưa đạt MaxIter)
 - a). Tính gradient $\nabla f(x_t)$.
 - b). Tính ma trận hessian $H = \nabla^2 f(x_t)$.
 - c). Tính ma trận nghịch đảo H^{-1}
 - d). Cập nhật tham số $x_{t+1} = x_t - H^{-1} \nabla f(x_t)$
3. End
4. Trả về giá trị x tối ưu
5. Kết thúc

2.3.1 Thuật toán Newton

❖ Hạn chế:

- ✓ Việc tính toán và nghịch đảo ma trận Hessian có chi phí tính toán lớn, đặc biệt với bài toán nhiều chiều.
- ✓ Khi Hessian suy biến hoặc có giá trị rất nhỏ, phép nghịch đảo trở nên không xác định hoặc gây mất ổn định số.
- ✓ Khi $\nabla^2 f(x_t)$ rất gần 0, thì giá trị $H^{-1}\nabla f(x_t)$ trở nên rất lớn, làm cho x_{t+1} di chuyển xa điểm khởi tạo ban đầu, có thể làm thuật toán mất ổn định.

2.3.1 Thuật toán Newton

❖ **Ví dụ 2** Tìm cực tiểu của hàm số

$$f(x_1, x_2) = x_1^4 + x_2^4 - 4x_1^2 - 4x_2^2, \text{ trong đó } x_1 \in [-3, 3] \text{ và } x_2 \in [-3, 3]$$

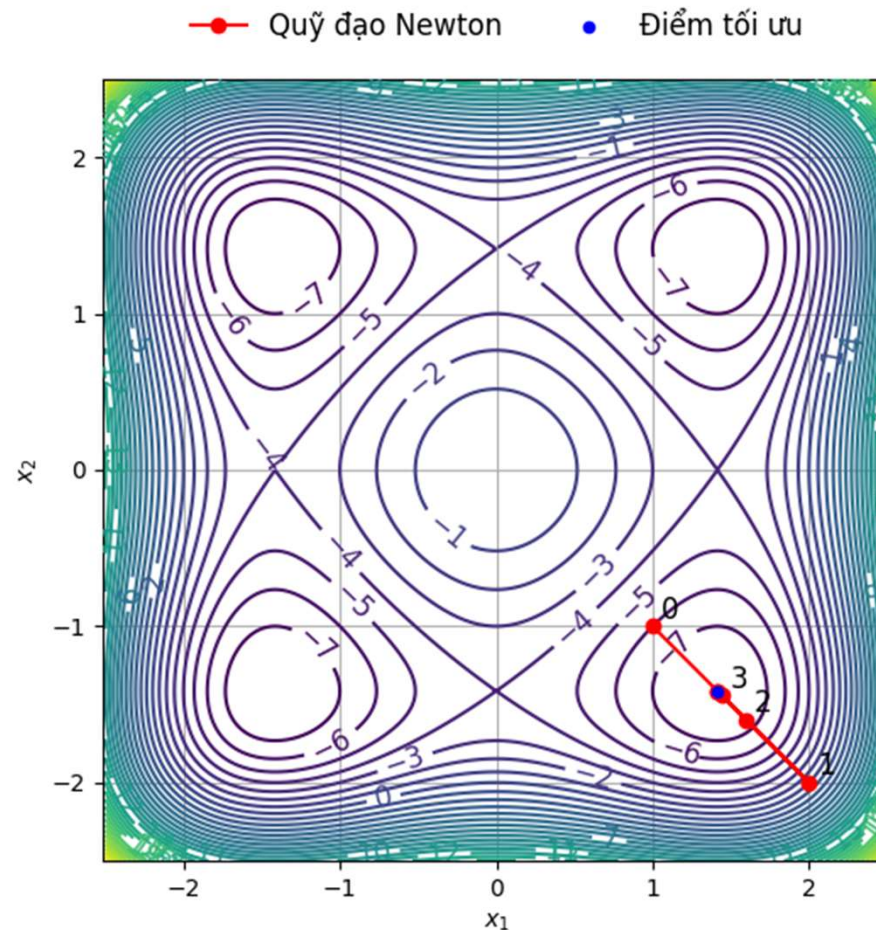
Gợi ý giải tay:

2.3.1 Thuật toán Newton

❖ **Ví dụ 2** Tìm cực tiểu của hàm số

$$f(x_1, x_2) = x_1^4 + x_2^4 - 4x_1^2 - 4x_2^2, \text{ trong đó } x_1 \in [-3, 3] \text{ và } x_2 \in [-3, 3]$$

Chạy code:



2.3.2 Thuật toán Quasi-Newton

❖ **Ý tưởng:** Phương pháp Quasi-Newton là một biến thể của phương pháp Newton, giúp tối ưu hóa nhanh nhưng không cần tính ma trận Hessian nghịch đảo H^{-1} trực tiếp mà thực hiện xấp xỉ nó.

❖ **Cập nhật**
$$x_{t+1} = x_t - Q_t^{-1} \nabla f(x_t)$$

✓ Q_t^{-1} là ma trận xấp xỉ của H^{-1}

❖ **Cập nhật ma trận xấp xỉ (BFGS)**

$$Q_{t+1} = \left(I - \frac{\delta_t \gamma_t^T}{\gamma_t^T \delta_t} \right) Q_t \left(I - \frac{\gamma_t \delta_t^T}{\delta_t^T \gamma_t} \right) + \frac{\delta_t \delta_t^T}{\gamma_t^T \delta_t}$$

✓ $\delta_t = x_{t+1} - x_t$ là bước cập nhật tham số.

✓ $\gamma_t = \nabla f(x_{t+1}) - \nabla f(x_t)$: chênh lệch gradient.

❖ **Nhận xét**

✓ Quasi-Newton hội tụ nhanh hơn GD.

✓ Không yêu cầu tính Hessian → phù hợp với bài toán quy mô lớn.

2.3.3 Thuật toán Levenberg-Marquardt

❖ Ý tưởng:

- ✓ Levenberg–Marquardt là phương pháp tối ưu phi tuyến kết hợp Newton và GD bằng cách thêm hệ số suy giảm vào Hessian, giúp cân bằng giữa hội tụ nhanh và ổn định.
- ✓ LM thường được sử dụng trong các bài toán tối ưu không lồi, khi ma trận Hessian suy biến hoặc không xác định dương, khiến phương pháp Newton dễ mất ổn định, trong khi Gradient Descent không đạt hiệu quả mong muốn.

2.3.3 Thuật toán Levenberg-Marquardt

❖ **Cập nhật:** $x_{t+1} = x_t - (H + \delta I)^{-1} \nabla f(x_t)$

- ✓ $H = \nabla^2 f(x_t)$ là ma trận đạo hàm riêng bậc 2 của hàm f
- ✓ I là ma trận đơn vị.
- ✓ δ điều chỉnh mức độ ảnh hưởng của ma trận Hessian.

❖ Cơ chế hoạt động

- ✓ Khi δ nhỏ, phương trình này giống phương pháp Newton, tận dụng đầy đủ thông tin Hessian để hội tụ nhanh hơn.
- ✓ Khi δ lớn, bước cập nhật giống gradient descent với hệ số học $\alpha \approx \frac{1}{\delta}$ giúp tăng tính ổn định nhưng chậm hơn

❖ Hạn chế

- ✓ Chi phí tính toán cao do cần tính và nghịch đảo ma trận Hessian
- ✓ Yêu cầu bộ nhớ lớn, không phù hợp bài toán có số chiều rất cao.

THANK YOU!